

Strang-Splitting Online Gate with $O(D)$ Memory Estimator

This document consolidates the **second-order (Strang) split-step stability gate** for a Π -kernel swarm with:

- **Expander model:** LPS Ramanujan family, fixed degree $\mathbf{d} = 12$ (choose $p = 11$)
- **Spectral constant:** $\sigma = 2\sqrt{d-1}/d = 2\sqrt{11}/12 \approx 0.552771$, so $1 - \sigma \approx 0.447229$
- **Estimator settings:** $\mathbf{m} = 8$ probes, EWMA window $\mathbf{W} = 19 \Rightarrow \beta = 0.9$, safety inflation $\chi = 1.5$

The goal is a **single scalar gate** that can be enforced online:

$$\frac{\Delta t^3}{8} \chi \tilde{C}_{\oplus}(t) \leq \alpha \Big[\lambda \gamma + (1 - \lambda \gamma) \eta (1 - \sigma) \Big]$$

where $\tilde{C}_{\oplus}(t)$ is a smoothed (EWMA) estimate of the nested-commutator magnitude.

1) Objects and definitions

Operators

Let H_1, H_2 be linear operators (typically Hermitian generators). Define nested commutators:

$$K_1 := [H_1, [H_1, H_2]], \quad K_2 := [H_2, [H_2, H_1]].$$

Expanded forms (useful for computation):

$$K_1 = H_1^2 H_2 - 2H_1 H_2 H_1 + H_2 H_1^2,$$

$$K_2 = H_1 H_2^2 - 2H_2 H_1 H_2 + H_2^2 H_1.$$

Contraction slack (expander-dependent)

Let $\kappa = 1 - \lambda\gamma$. Using lazy mixing $A(\eta) = (1 - \eta)I + \eta P$, the expander disagreement contraction margin contributes the factor $\eta(1 - \sigma)$.

Define the **available slack**:

$$S(\alpha, \eta, \lambda, \gamma) := \alpha \Big[\lambda \gamma + (1 - \lambda \gamma) \eta (1 - \sigma) \Big].$$

For $\mathbf{d} = 12$ Ramanujan expanders:

$$1 - \sigma \approx 0.447229, \quad S = \alpha \Big[\lambda \gamma + 0.447229 \sqrt{\eta(1 - \lambda \gamma)} \Big].$$

2) Single-scalar commutator estimator with shared probes

We estimate

$$|K_1|_F^2 + |K_2|_F^2.$$

Shared-probe Hutchinson estimator

Let probes z_k satisfy $\mathbb{E}[z_k z_k^\dagger] = I$ (Rademacher/Steinhaus/complex normal). With $\mathbf{m} = 8$ probes:

$$\widehat{S}(t) := \frac{1}{8} \sum_{k=1}^8 \big(|K_1 z_k|_2^2 + |K_2 z_k|_2^2 \big) \approx |K_1|_F^2 + |K_2|_F^2.$$

Define $\widehat{C}_\oplus(t) := \sqrt{\widehat{S}(t)}$.

EWMA smoothing ($\beta = 0.9$)

Smooth the **squared** quantity:

$$\widetilde{S}(t) = 0.9 \widetilde{S}(t-1) + 0.1 \widehat{S}(t), \quad \widetilde{C}_\oplus(t) = \sqrt{\widetilde{S}(t)}.$$

Initialize $\widetilde{S}(0) = \widehat{S}(0)$.

3) The Strang gate (single inequality)

With $\chi = 1.5$:

$$\boxed{\frac{\Delta t^3}{(1.5) \widetilde{C}_\oplus(t)} \leq \alpha \Big[\lambda \gamma + 0.447229 \sqrt{\eta(1 - \lambda \gamma)} \Big]}$$

Equivalently (since $1.5/8 = 0.1875$):

$$\boxed{0.1875 \Delta t^3 \widetilde{C}_\oplus(t) \leq \alpha \Big[\lambda \gamma + 0.447229 \sqrt{\eta(1 - \lambda \gamma)} \Big]}$$

If violated, shrink to the boundary:

$$\Delta t \rightarrow \min(\Delta t, \frac{8}{1.5} \cdot \frac{S(\alpha, \eta, \lambda, \gamma)}{|\widetilde{C}_{\text{oplus}}(t)|^{1/3}}) = \min(\Delta t, 5.33333 \cdot \frac{S}{|\widetilde{C}_{\text{oplus}}|^{1/3}})$$

4) $O(D)$ peak-memory estimator: scan over probes

When even a batch $(D, 8)$ is too large, compute each probe sequentially with peak memory $O(D)$.

Minimal-apply per-probe kernel (12 applies)

```

# Pseudocode: single probe,  $O(D)$  memory
# H1_one, H2_one:  $(D, ) \rightarrow (D, )$ 

def one_probe_score(H1_one, H2_one, z):
    # 1-6 shared intermediates
    z1 = H1_one(z)      # (1) H1 z
    z2 = H2_one(z)      # (2) H2 z

    z12 = H1_one(z2)    # (3) H1 H2 z
    z21 = H2_one(z1)    # (4) H2 H1 z

    z11 = H1_one(z1)    # (5) H1^2 z
    z22 = H2_one(z2)    # (6) H2^2 z

    # K1 z = H1^2 H2 z - 2 H1 H2 H1 z + H2 H1^2 z
    tmp = H1_one(z12)    # (7) H1^2 H2 z
    tmp = tmp + H2_one(z11) # (8) + H2 H1^2 z
    tmp = tmp - 2.0 * H1_one(z21) # (9) - 2 H1 H2 H1 z
    s1 = (tmp.conj() @ tmp).real

    # K2 z = H1 H2^2 z - 2 H2 H1 H2 z + H2^2 H1 z
    tmp = H1_one(z22)    # (10) H1 H2^2 z
    tmp = tmp + H2_one(z21) # (11) + H2^2 H1 z
    tmp = tmp - 2.0 * H2_one(z12) # (12) - 2 H2 H1 H2 z
    s2 = (tmp.conj() @ tmp).real

    return s1 + s2

```

Scan over $m=8$ probes

```

# Z: (m, D) probes, m=8

def estimate_S_hat_scan(H1_one, H2_one, Z):
    total = 0.0

```

```

for z in Z:
    total += one_probe_score(H1_one, H2_one, z)
return total / Z.shape[0]

```

EWMA + gate integration

```

# fixed: beta=0.9, chi=1.5

S_hat = estimate_S_hat_scan(H1_one, H2_one, Z) # scalar
S_tilde = 0.9 * S_tilde + 0.1 * S_hat
C_tilde = sqrt(S_tilde)

slack = alpha * (lambda_*gamma + 0.447229*eta*(1 - lambda_*gamma))

if 0.1875 * (dt**3) * C_tilde > slack:
    dt = min(dt, (5.333333 * slack / C_tilde) ** (1/3))

```

5) Batched micro-optimized estimator (low intermediates, returns S_{hat} only)

When $(D, 8)$ fits, batching is typically faster (BLAS/FFT/block-Krylov gains).

```

# Z: (D, m) columns are probes (m=8)
# H1, H2: batched apply functions (D,m)->(D,m)

def col_norm2(X):
    return (abs(X)**2).sum(axis=0) # (m,)

def estimate_S_hat_strang_batched(H1, H2, Z):
    # Shared intermediates
    Z1 = H1(Z) # (1)
    Z2 = H2(Z) # (2)
    Z12 = H1(Z2) # (3)
    Z21 = H2(Z1) # (4)
    Z11 = H1(Z1) # (5)
    Z22 = H2(Z2) # (6)

    # K1 part (tmp reused; no K1Z stored)
    tmp = H1(Z12) # (7)
    tmp = tmp + H2(Z11) # (8)
    tmp = tmp - 2.0*H1(Z21) # (9)
    s = col_norm2(tmp)

```

```

# K2 part
tmp = H1(Z22)          # (10)
tmp = tmp + H2(Z21)    # (11)
tmp = tmp - 2.0*H2(Z12) # (12)
s = s + col_norm2(tmp)

return s.mean()

```

6) Apply-count and cost table (this schedule)

Fixed apply counts (per probe)

- Total operator applies: **12** per probe
- Split: **6** calls to H_1 , **6** calls to H_2
- Plus vector $AX+Y$ and two dot products: $O(D)$ extra

Complexity comparison

Operator type	One apply cost (rough)	Per-probe cost here	Notes
Dense Hv	$\Theta(D^2)$	$12 \Theta(D^2)$	Batch $(D, 8)$ uses GEMM for speed; scan uses GEMV.
Sparse SpMV	$\Theta(\text{nnz})$	$6 \Theta(\text{nnz}_1) + 6 \Theta(\text{nnz}_2)$	Usually bandwidth-bound; avoid composite products (densify).
FFT-based $F^{-1}\Lambda F$	$\approx 2c D \log D + O(D)$	~ 12 FFTs per probe (since 6 H_1 applies $\times$ (FFT+IFFT))	Diagonal H_2 is $O(D)$. Use rFFT if real.
Krylov solve $A^{-1}v$	$\approx k(C_{\text{matvec}} + C_M)$	$6k_1(\cdot) + 6k_2(\cdot)$	Batch enables block-Krylov; scan loses block benefits but saves memory.

7) Example operator apply patterns

(a) Dense

```

def H1(X): return H1_mat @ X

def H2(X): return H2_mat @ X

```

(b) Sparse

```
def H1(X): return H1_sp @ X

def H2(X): return H2_sp @ X
```

(c) FFT-based (kinetic + potential)

```
def H1(X):
    Xk = fft(X, axis=0)
    Yk = k_hat[:, None] * Xk
    return ifft(Yk, axis=0)

def H2(X):
    return V[:, None] * X
```

(d) Black-box Krylov (implicit inverse)

```
def H1(X): return solve_block(A, X, M=precondA)

def H2(X): return solve_block(B, X, M=precondB)
```

References (background)

1. Lubotzky, Phillips, Sarnak. *Ramanujan Graphs* (construction of LPS graphs).
2. Lubotzky. *Discrete Groups, Expanding Graphs and Invariant Measures* (expander background).
3. Hairer, Lubich, Wanner. *Geometric Numerical Integration* (splitting methods; Strang error terms via nested commutators).
4. Hutchinson. *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines* (randomized trace estimators).
5. Avron, Toledo. *Randomized algorithms for estimating the trace of an implicit symmetric positive semidefinite matrix* (variance bounds/modern view).